

IntSeqBERT: Learning Arithmetic Structure in OEIS via Modulo-Spectrum Embeddings

Kazuhisa Nakasho

CICM 2026

Iwate Prefectural University, Japan

Code & data: github.com/aabaa/IntSeqBERT

Why Finding Good Problems Matters

Terence Tao's concern: We may solve fruitful open problems faster than we develop new ones.

“a country or region can suffer a critical shortage of drinking water while simultaneously being surrounded by a massive ocean.”

- We can ask **infinitely many mathematical questions**.
- But **good problems may still become scarce**: many questions lead to few new ideas or connections.
- A useful problem should help us **understand more mathematics**.

Our motivation

Can AI help us find new mathematical connections and promising conjectures?

Source: Terence Tao, Mastodon, post 1/4. Quote above; other text is a summary.

mathstodon.xyz/@tao/117237320796901560

Long-Term Goal — AI for Mathematical Discovery

- Many important conjectures began with a **numerical observation**:
 - $196,884 = 196,883 + 1$ — McKay's observation $\rightarrow$ *Monstrous Moonshine*
 - The BSD conjecture and mirror symmetry (317,206,375 rational curves) also began with numerical patterns
- Today, AI systems take part in this kind of discovery:
AlphaProof, the **Ramanujan Machine**, **FunSearch**
- **Our long-term goal**: an AI that finds *hidden number-theoretic similarities* between integer sequences from different fields — and supports **conjecture generation**
- Our data source is the **OEIS**: 391,710 sequences, each with a precise definition — a collection of mathematics that computers can read
- We need a machine **representation** that captures number-theoretic structure
- **This paper is the first step**: we propose such a representation, and we *measure* how much arithmetic structure it actually captures

Why Multiplication Is Hard for Deep Learning

- The main problem: standard neural models have difficulty learning **multiplication-like rules**.
- **Spectral bias**: networks learn low-frequency, addition-like structure first (Rahaman+ 2019)
- How many layers does multiplication need?
 - **In theory**: **depth-3 threshold circuits** are enough for n -bit multiplication, and fewer layers are not enough (Siu & Roychowdhury 1994)
 - **In practice**: gradient descent does not find these circuits — GPT-4 scores 59% on 3×3 -digit, and only **4% on 4×4 -digit** multiplication (Dziri+ 2023)
 - Even **mod- p multiplication in a 2-layer Transformer** generalises only after a very long training period — the *grokking* phenomenon (Power+ 2022)
 - One successful method uses **12 layers *plus* a special training method** (implicit chain-of-thought) to reach 15-digit products (Yao+ 2025)
- **Tokenisation makes it worse**: values outside the vocabulary become UNK, and token IDs do not directly show arithmetic structure
- The OEIS is a **difficult test**: its values range from single digits ... to 10^{210}

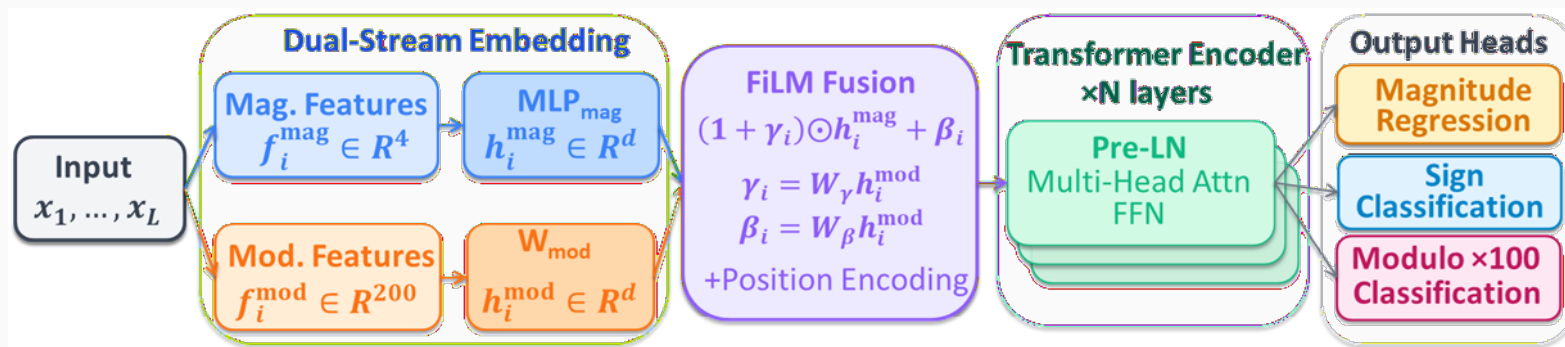
Key Idea — Show Multiplicative Structure in the Input

- Our key idea comes from **classical number theory**.
- Reduction mod m is a **ring homomorphism** $\mathbb{Z} \rightarrow \mathbb{Z}/m\mathbb{Z}$: it preserves **both addition and multiplication**
- So the residue sequence $(x_i \bmod m)$ keeps the multiplicative structure — **even for very large values**: for 10^{210} , the residue stays in $\{0, \dots, m-1\}$
- Number theory explains these **periodic patterns**:
 - **Fermat's little theorem**: powers are periodic mod p (period divides $p-1$)
 - **Quadratic reciprocity**: links residue information across primes
 - **CRT**: composite moduli combine information from several prime powers
- **Hypothesis**: give the network the *modulo spectrum* as input — then it sees multiplicative structure **at depth zero**, without having to learn it across many layers

Task — Masked Sequence Modelling on OEIS

- The task is **masked sequence modelling** — BERT-style, but for integers.
- We take up to the first 128 terms of an OEIS sequence, mask 15% of the positions, and train the model to predict each masked value from context
- This tests how well the model has learned the *rule* of the sequence; **next-term prediction** is the special case where the last term is masked
- Prior benchmark: **FACT** (NeurIPS 2022) — a plain Transformer on five OEIS tasks; *unmasking* was the hardest; our **Vanilla baseline** uses that architecture
- Instead of predicting one token, we predict **three quantities** per masked position:
 1. **Magnitude**: $v_i = 1 + \log_{10} |x_i|$
 2. **Sign**: $s_i \in \{+, -, 0\}$
 3. **Residues**: $x_i \bmod m$ for every $m \in \{2, \dots, 101\}$

IntSeqBERT — Dual-Stream Architecture



- The **magnitude stream** ($\mathbb{R}^4$) encodes log-scale size and sign $\rightarrow$ MLP
- The **modulo stream** ($\mathbb{R}^{200}$) embeds each residue on the **unit circle**: $\phi_m(r) = [\sin(2\pi r/m), \cos(2\pi r/m)]$ — we use the Transformer's sinusoidal *positional* encoding for *residues of values*; residues $m-1$ and 0 stay neighbours
- **FiLM fusion**: the modulo stream scales and shifts the magnitude stream: $\mathbf{e}_i = (1 + \gamma_i) \odot \mathbf{h}_i^{\text{mag}} + \beta_i$
- Then a standard **pre-LN Transformer encoder** — Small 6.4M / Middle 29.0M / **Large 91.5M** parameters

Training — Multi-Task Objective

- Three prediction heads use the encoder output:
 - The **magnitude head** also predicts its own **uncertainty** (μ_i, σ_i^2) — remember this: the Solver will use it
 - The **sign head**: 3-class classification
 - The **modulo head**: 100 independent classifiers, each normalised by its maximum entropy $\ln m$ — so mod 2 and mod 101 contribute fairly
- The loss is a weighted sum: $\mathcal{L} = \mathcal{L}_{\text{mag}} + \mathcal{L}_{\text{sign}} + 2 \mathcal{L}_{\text{mod}}$
- Trained on **219,765 OEIS sequences**, 200 epochs, in **FP32** — because values near 10^{210} overflow FP16
- All experiments run on a **single consumer GPU** (RTX 3070 Ti, 8 GB), so other researchers can repeat the experiments

Neural-Symbolic Decoding — the CRT Solver

- The network outputs **distributions** — magnitude, sign, 100 residues — **not integers**
- The **Solver** produces integer candidates using **symbolic rules**:
 - The magnitude and its uncertainty give a 3σ **search interval**
 - **Dense** ($\Delta n \leq 10^6$): check every candidate in the interval
 - **Sieve** ($\leq 10^{14}$): CRT beam search based on the moduli with the most confident predictions
 - **CRT** ($> 10^{14}$): sparse CRT generates very large candidates directly
- We combine neural prediction and symbolic rules in a **neural-symbolic** system — residue arithmetic and the Chinese Remainder Theorem are **built in**; the network does not have to learn these rules from data
- Is this a strength or a limitation? — **I will come back to this with the results.**

Dataset — 274,705 OEIS Sequences

- From 391,710 OEIS entries we keep **274,705 (70.1%)**, split 8:1:1
- Eleven tags are excluded, in **three groups**:
 - **depend on how values are written** (cons, cofr, frac, base, word): their patterns depend on the *encoding* of the values
 - **do not fit this task** (fini, tabl)
 - **quality labels** (dead, unkn, less, dumb)
- These choices define our task: less/dumb are subjective labels, and tabl could be converted into sequences. Future benchmarks may use different choices
- For evaluation, five **magnitude buckets** (groups by size):
Small ($< 10^2$, 52.7%) → Medium → Large → Huge → Astronomical ($\geq 10^{50}$)

Metrics — and Three Questions

- **Magnitude Accuracy ($\text{Acc}_{0.5}$) is NOT exact match**: it asks whether the prediction is within 0.5 on the log scale — a factor of $\sqrt{10} \approx 3.16$
 - Example: true value $10^6 \rightarrow$ anything in $[3.2 \times 10^5, 3.2 \times 10^6]$ counts
 - Its purpose: help set a **realistic search range** for the Solver
- **Exact** integer match is measured separately, by **Solver Top- k**
- **MMA**: mean classification accuracy over all 100 moduli
- **NIG** (normalised information gain) $= 1 - \mathcal{L}_{\text{CE}}^{(m)} / \ln m$: 0 = no gain over uniform prediction, 1 = perfect prediction

Three questions for the results

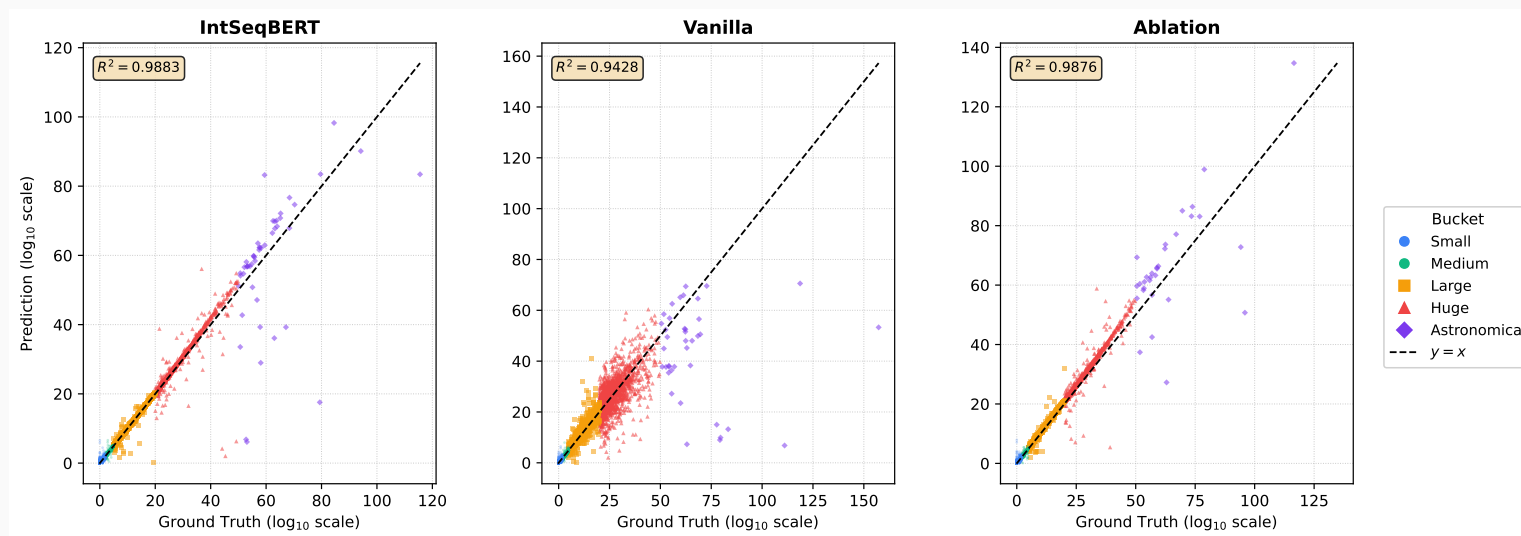
Q1 — Does the dual stream help? **Q2** — Where does it help? **Q3** — Can we predict exact integers?

Q1 — Does It Help? Yes, Consistently

Size	Model	Mag Acc	Sign Acc	MMA
Large	IntSeqBERT	95.85	98.54	50.38
Large	Vanilla	86.97	97.66	45.85
Large	Ablation (no modulo)	89.70	98.29	35.22

- At the Large scale, IntSeqBERT reaches **95.85%** magnitude accuracy — **+8.9 pt** over the tokenised baseline — and **50.38%** mean modulo accuracy (**+4.5 pt**)
- The **ablation row** is the important one: remove the modulo stream → MMA drops **−15.2 pt** — and magnitude accuracy **also** drops **−6.2 pt**
- The residues limit the possible integer values. This helps the model learn **both magnitude and modulo information**
- (The same pattern holds at Small and Middle scale — scaling results in backup)

Q2 — Where? Exactly Where Tokenisation Breaks



MSE	IntSeq	Vanilla	Abl.
Small	0.111	0.138	0.103
Large	0.162	2.100	0.381
Huge	2.082	22.73	5.021

- Above its vocabulary limit, Vanilla uses UNK — its Large-bucket error is **13× ours**; overall R^2 : **0.988** vs 0.943
- This is why bucketed evaluation matters: most OEIS sequences grow slowly, so easier cases have a large effect on the *overall* scores
- The clearest gains are in the **hard buckets** — **Large and above** — where the baseline stops working

Q3 — The Strictest Test: Exact Next-Term Prediction

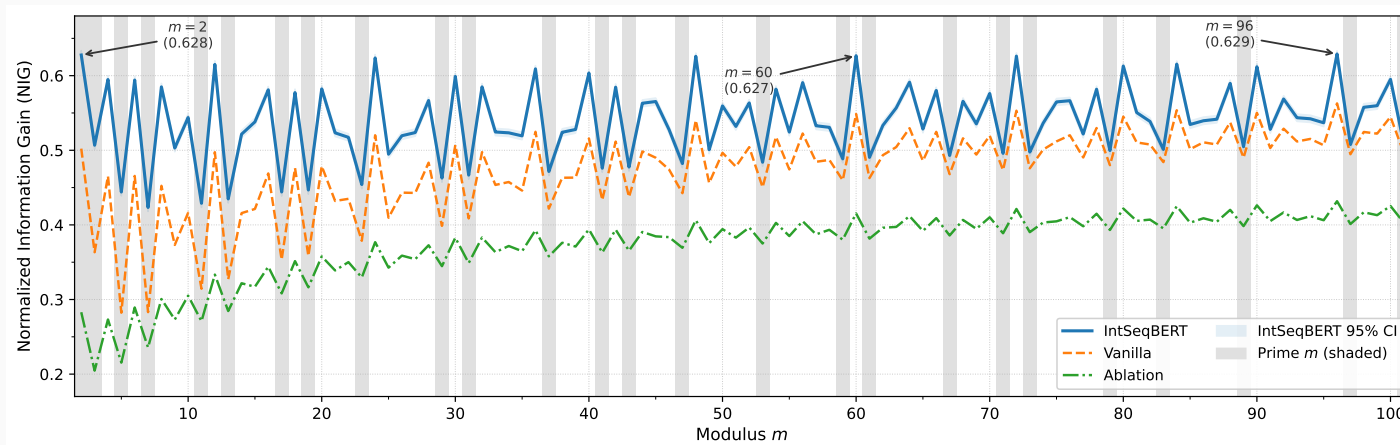
Size	Model	Top-1	Top-10
Large	IntSeqBERT	19.09	26.23
Large	Vanilla	2.59	3.80
Large	Ablation	11.75	21.79

- We test the full system: predict the **exact last term** of 10,000 sequences kept separate for testing
- The tokenised Transformer gets **2.59%**; IntSeqBERT gets **19.09%** — a **7.4-fold improvement**
- By bucket: Small **68.3%**, Medium **20.8%** — while Vanilla is at **0%** for everything beyond its vocabulary
- But for the Large buckets and above, our model also drops to near zero: CRT reconstruction needs **exact residues for many moduli at the same time** — and at MMA $\approx 50\%$, that almost never happens
- The next question: **is 19% a success or a failure?**

What Does 19% Mean? — A Useful Negative Result

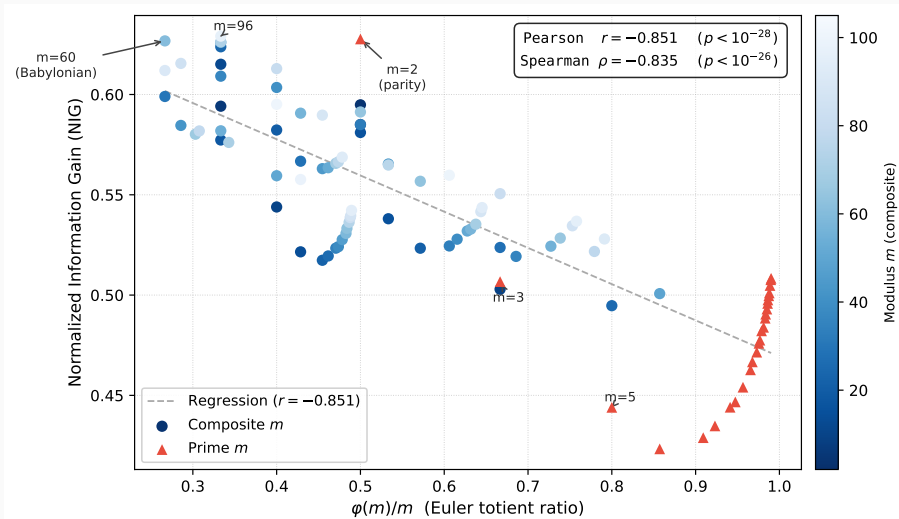
- There are **two useful ways to understand this result**.
- Even with number-theoretic structure **built into** the representation, exact prediction of OEIS terms **remains hard**; a purely tokenised Transformer performs poorly (2.59%)
- **Point 1**: the results measure **how limited end-to-end learning is** for arithmetic in this setting
- **Point 2**: the results support **neural-symbolic design** — the CRT solver provides exact rules that the network does not learn well on its own
- And my answer to the earlier question — strength or limitation?
We use features based on number theory as a **tool for measurement**: removing the modulo stream shows how much these features help the model learn arithmetic structure (−15.2 pt MMA)

But What Did the Network Learn? The NIG Spectrum



- Prediction accuracy is one way to evaluate the model. We also ask: **what did the network actually learn about number theory?**
- NIG for every modulus $m = 2, \dots, 101$: IntSeqBERT is above both baselines across the **entire spectrum** — and the shape is not flat: **the peaks follow a clear pattern**
- Best modulus across *all* models and scales: $m = 96 = 2^5 \times 3$ (NIG = 0.629); second best: $m = 2$ — **parity is everywhere** in the OEIS
- Parity also shows differences between the models: mod-2 accuracy **85.7** (IntSeq) / 81.4 (Vanilla) / 72.1 (Ablation)

Finding — Higher NIG with a Lower Totient Ratio



- Why 96? Why composites? One quantity helps explain this pattern: Euler's totient ratio
$$\varphi(m)/m = \prod_{p|m} (1 - 1/p)$$
- NIG vs $\varphi(m)/m$: Pearson $r = -0.851$ ($p < 10^{-28}$)
- We explain this using the CRT aggregation effect: $x \bmod 96$ gives us $x \bmod 2, 4, 8, 16, 32$ — and $x \bmod 3$ — at the same time
- This is an experimental result, not a theorem. It suggests a **design choice**: to extend beyond 101, add highly composite numbers, not more primes
- This spectrum describes the arithmetic patterns in a dataset. We can use spectra to compare sequences or datasets

Positioning in Computer Mathematics

- Comparing sequences brings us back to our original goal: **what is this work for?**
- **Program synthesis on the OEIS** — Alien Coding, QSynt, Learning Conjecturing — describes each sequence's **generation rule**
- Our work can **support these methods**: we learn **shared arithmetic structure** across the whole corpus
 - Synthesis asks: *“What program generates this sequence?”*
 - Our representation asks: *“Which sequences are number-theoretically **similar** — even if generated in completely different ways?”*
- Pretrained embeddings could **guide the search** in program synthesis, or help us **find similar sequences**
- Finding similar sequences could help us **generate conjectures** — our long-term goal
- Code, preprocessing, and evaluation: github.com/aabaa/IntSeqBERT

Conclusion

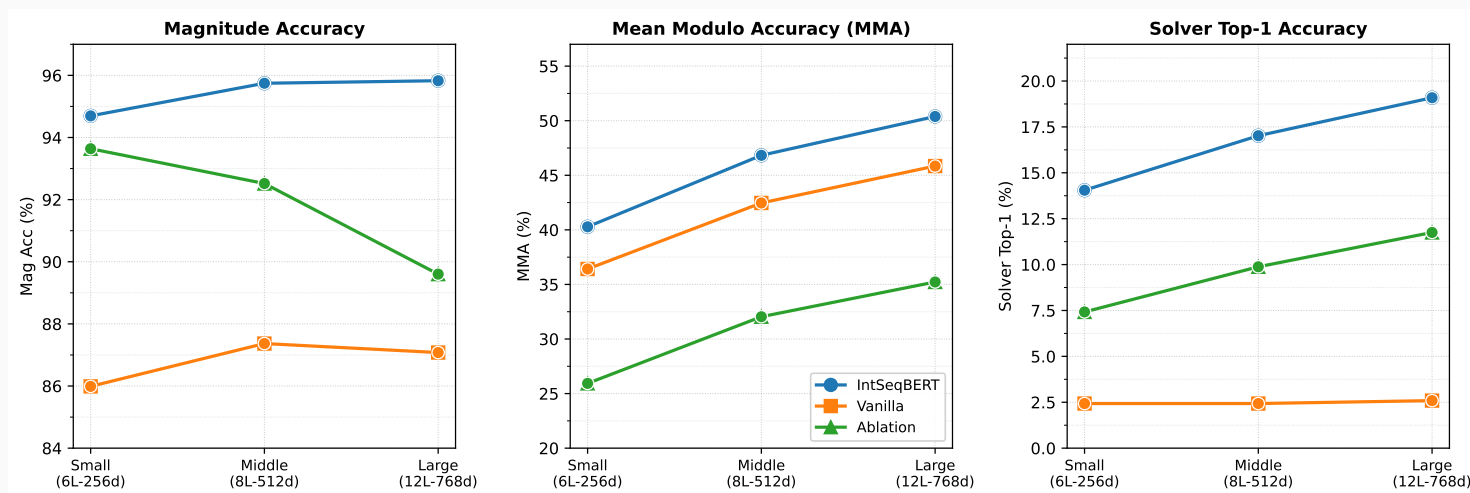
Four main points:

1. The modulo spectrum uses features **based on number theory**: a **ring homomorphism** makes multiplicative structure visible **at depth zero**
2. It works: consistent gains at every scale, and a **7.4×** improvement in exact next-term prediction
3. **Composite moduli combine arithmetic information**: NIG has a negative correlation with $\varphi(m)/m$ ($r = -0.851$)
4. At the same time: pure neural prediction of OEIS terms **remains hard** — this supports further work on **neural-symbolic methods**

Future work: approximate CRT for large integers · highly composite moduli beyond 101 · larger-scale retraining · **similarity retrieval & conjecture generation**

Thank you! — github.com/aabaa/IntSeqBERT

Backup — Full Results & Scaling Behaviour



- The same ranking holds at every scale: IntSeqBERT > Vanilla > Ablation
- **Model size helps modulo prediction more**: from Small to Large, MMA improves **+10.0 pt** while Mag Acc improves only **+1.1 pt** — larger models improve modulo prediction more than magnitude prediction
- Solver Top-1 also improves: **+5.0 pt** (14.05 → 19.09)
- No clear sign of overfitting: Train 1.00 / Val 1.01 at epoch 200; larger-scale retraining is planned as future work

Backup — Why Not Digit-Based Tokenisation?

- A natural alternative: treat each decimal digit as a token, and generate digit by digit
- We have not made an experimental comparison in this paper — it is future work
- Our target is not the pattern of decimal digits, but the **number-theoretic structure of the integer itself**:
 - the modulo stream supplies residues **directly**; with digit tokens, the model must **calculate** them from the digits
- Digit-by-digit generation also **changes the problem setting**: sequential prediction of a digit string vs. predicting one integer value
- We plan to compare the two settings under the same conditions

Backup — Limitations

- **Large integers:** CRT reconstruction needs exact residues for many moduli at the same time; 13.4% of Solver calls return no valid candidate (none mode); approximate CRT / less strict residue constraints are possible next steps
- **Dataset bias:** many nonn sequences → high sign accuracy; many slowly growing sequences → high overall Mag Acc
- **Compute:** a single 8 GB GPU, single seed (42), vocabulary 20,003 — Vanilla uses the *same architecture as FACT* with our settings. We do not reproduce FACT's reported results

Backup — How Hard Is Multiplication for Neural Networks?

- **Circuit theory:** n -bit multiplication is **not in AC^0** (constant-depth AND/OR), but **is** computable by polynomial-size **depth-3 threshold circuits** — and depth 3 is optimal (Siu & Roychowdhury, *SIAM J. Discrete Math.*, 1994); iterated products: depth 4
- **Transformers $\approx TC^0$:** log-precision transformers are upper-bounded by uniform TC^0 (Merrill & Sabharwal 2023) — this connects multiplication to the limits of these models
- **LLM experiments** (Dziri+, *Faith and Fate*, NeurIPS 2023): GPT-4 zero-shot — 3×3 -digit: **59%**, 4×4 -digit: **4%**
- **Grokking** (Power+ 2022, arXiv:2201.02177): a 2-layer transformer on modular arithmetic generalises only after a very long training period
- **Mechanistic analysis** (arXiv:2510.00184, 2025): standard training fails at multi-digit multiplication even at 90B parameters; a 12-layer model succeeds at 15-digit products only with implicit-CoT training, forming a **binary tree of partial products** in attention